fmxData

PHP Class for the FileMaker Data API
Documentation

Version 0.3.1

Author: Duane Weller (eXcelisys Inc.)

©2026 Duane Weller

CONTENTS

WHAT IS FMXDATA?

WHY FMXDATA?

SUPPORTED DATA API FEATURES

THINGS NOT SUPPORTED

PREPARING FILEMAKER SERVER

INSTALLING THE EXAMPLE FILES

PREPARING AND HOSTING YOUR FILEMAKER FILE

ADDING THE LIBRARY TO YOUR PROJECT

CONNECTING TO FILEMAKER SERVER

- fmxConnect ($host, $database, $username, $password {, $sslVarify })

DISCONNECTING FROM FILEMAKER SERVER

- fmxDisconnect ( )

FILEMAKER SERVER FUNCTIONS

- fmxProductInfo ( )
- fmxListDatabases ( )

DATABASE FUNCTIONS

- fmxListScripts ( )
- fmxListLayouts ( )
- fmxLayoutMeta ($layout {, $recordId })

RECORD FUNCTIONS & OPTIONS

- fmxCreateRecord ($layout)
- fmxEditRecord ($layout, $recordId)
- fmxDeleteRecord ($layout, $recordId)
- fmxDuplicateRecord ($layout, $recordId)
- fmxGetRecord ($layout, $recordId)
- fmxGetRange ($layout)

- fmxFindRecords($layout)

SETTING FIELDS IN FILEMAKER

- addPostFieldArray($fieldArray)
- addPostField($fieldName, $fieldValue)

PERFORMING FILEMAKER FINDS

- addRequestArray($requestArray {, $omit })

SORTING THE RESULTS

- addSortParam($fieldName {, $direction, $sortOrder})

LIMITING THE FIND RESULTS

- setLimitOffset($limit {, $offset})

WORKING WITH FILEMAKER SCRIPTS

- addScriptPrerequest($scriptName {, $scriptParameter)
- addScriptPresort($scriptName {, $scriptParameter)
- addScript($scriptName {, $scriptParameter)

SETTING THE RESPONSE LAYOUT

- setResponseLayout($layout)

WORKING WITH PORTALS

- limitPortals($portalNames)
- limitPortalRows ($portalName, $limit {, $offset })

WORKING WITH CONTAINER FIELDS

- fmxUploadFile($pathToFile, $layout, $recordId, $fieldName {, $repetition })
- fmxContainerData($url)

SETTING GLOBAL FIELDS

- fmxSetGlobalFields( )
- addGlobalField($baseTable, $fieldName, $value)

WHAT IS FMXDATA?

FMXData is a PHP class that provides a library of simple functions for working with the FileMaker Data API. The PHP class takes care of creating the appropriate cURL calls to connect and run queries with the Data API, the full response is then returned to the calling script.

WHY FMXDATA?

FMXData is structured a little differently then some of the other PHP classes for the Data API. As a single file it offers a simple way to add a library of PHP functions to easily work with the Data API. Functions such as connecting, disconnecting, and uploading container data are static functions and do not require you to initialize a class instance to use. Yet, you can initialize separate instances for each query. This makes it much more flexible to use in PHP framework, CMS, or even just on a simple PHP web page.

SUPPORTED DATA API FEATURES

fmxData supports all of the following Data API features.

1. Log in to a database session.
2. Log out of a database session.
3. Get Product Information
4. Get Database Names
5. Get Script Names
6. Get Layout Names
7. Get Layout Metadata
8. Create a Record
9. Edit a Record
10. Delete a Record
11. Duplicate a Record
12. Get a Record
13. Get a Range of Records
14. Upload a File to a Container Field
15. Download a Container Field file
16. Find FileMaker Records
17. Set FileMaker Globals
18. Set a Response Layout
19. Perform a Script
20. Perform Pre-request Script
21. Perform Presort Script
22. Limit which Portals to Return
23. Set Portal Limits and Offsets
24. Set the Record Limit and Offset
25. Add a Sort Field, Sort Direction, and Sort Order

THINGS NOT SUPPORTED

fmxData does not yet support the following FileMaker Data API features.

1) Log in to an external data source.

2) Log in to a database session using an OAuth identity provider.

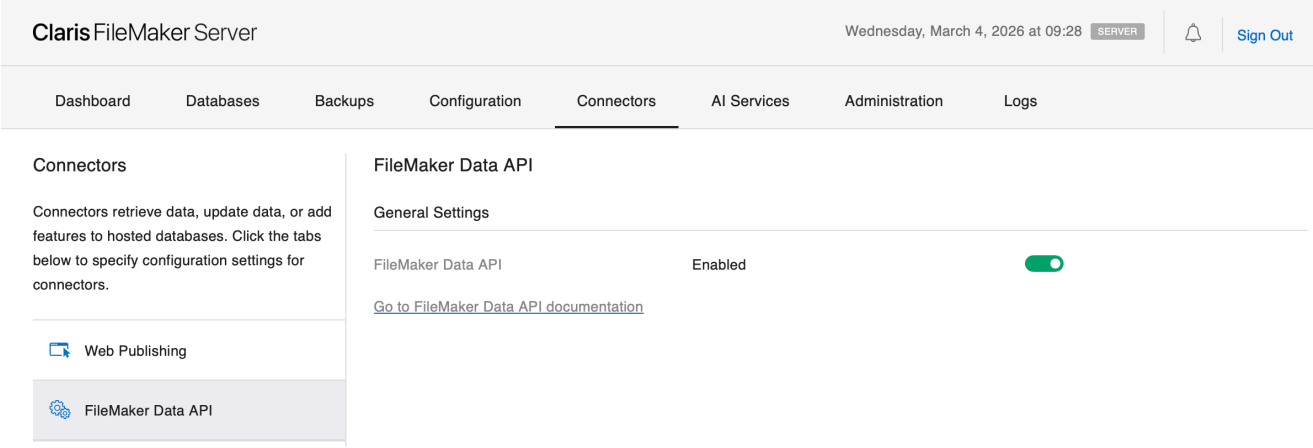
3) Log in to a database session using a FileMaker ID account.

4) Logging or Debugging

5) Checks or traps for FileMaker errors

The log in features were not necessarily needed for what I plan to use fmxData for but may be added in some future version. To keep this class lightweight I have not implemented any kind of logging or debugging. Some of the other PHP classes available offer those features. FMXData does not check or test for errors returned by the FileMaker Data API. The entire response is retuned to the calling script so that you can manage the FileMaker errors in your own PHP.

PREPARING FILEMAKER SERVER



Enable the FileMaker Data API
Go into the FileMaker Server Admin Console, under the connections area select the "FileMaker Data API" and enable it.

Hosting your PHP files
Since the PHP option was deprecated I've moved to a development environment that includes two computers, one to host FileMaker Server and the other to host my web server. This allows me to maintain a stock install of FileMaker Server that doesn't require configuration beyond the FileMaker Server Console. I run FileMaker Server by itself and run the web server on my development workstation. You will need to have a web server separate from the FileMaker Server to host your PHP files. Although I've never tried it, I don't see why you couldn't run FileMaker server in some sort of Virtual Machine (VM) on the same computer as your web server.

INSTALLING THE EXAMPLE FILES

1. Host the FMXData.fmp12 with FileMaker Server. Use the "Upload to Host" feature in FileMaker Pro Advanced (under "File -> Sharing -> Upload to Host") to upload the FMXData.fmp12 to the server.

Open the FMXData.fmp12 with the following credentials:

Username: admin
Password: admin

2. If you have not already done so, enable the FileMaker Data API in the FileMaker Server Admin Console. Under the connections area select the "FileMaker Data API" and enable it.

3. Set up and configure a web server on a separate computer from FileMaker Server. Install your web server of choice and PHP.

4. Add the example.php, FMXData.php, and photo.jpg files to the web server's root folder. The web server's root folder will depend on server you choose. Consult your server's documentation for the location.

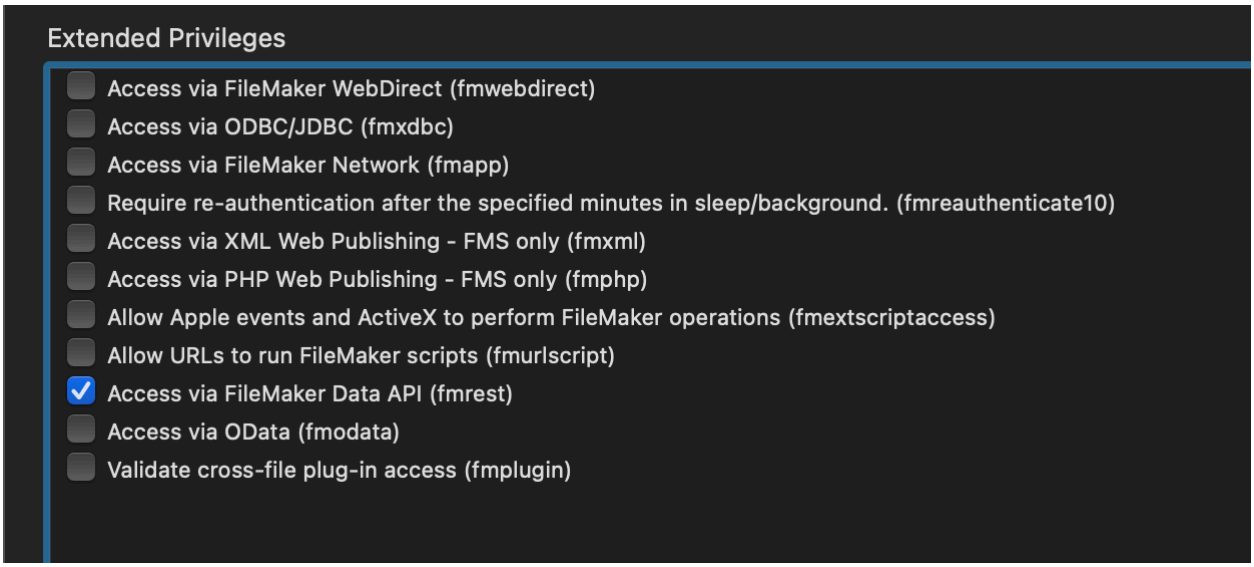
Accessing the Example Files from a Browser
Open a web browser and go to the following URL:

```
http://[localhost or web servers ip]/example.php
```

PREPARING AND HOSTING YOUR FILEMAKER FILE

NAMING - Try and keep your database, layout, script, and to some extent field names URL friendly. This means sticking to the standard alphanumeric characters. Avoid names with spaces and most special characters. Periods "." and dashes "-" are URL friendly but can't be used in FileMaker field names. The underscore "_" character is about the only special character that's universally acceptable.

SECURITY - I recommend you add a specific FileMaker Account for the Data API, however you can add the functionality to any FileMaker account by enabling the "Access via FileMaker Data API (fmrest)" extended privilege. This must be enabled before the account can connect using the FileMaker Data API.



ADDING THE LIBRARY TO YOUR PROJECT

To add the library to your project just require the FMXData file at the top of your PHP file.

Example Code:

```
require('FMXData.php');
```

CONNECTING TO FILEMAKER SERVER

Before you can start running finds and working with records you need to connect to the FileMaker database. Use the fmxConnect( ) function to create a session with FileMaker Server. Only one session may be open at a time. If you need to access more then one database or host, you can do it one at a time. Run the fmxDisconnect( ) function to close your fist session then run fmxConnect( ) to open a new session with the new database or host.

fmxConnect ($host, $database, $username, $password {, $sslVarify })

Parameters:
$host - The host server's domain or IP - can be localhost or 127.0.0.1 as well.
$database - The filename of the FileMaker file without the extension (.fmp12).
$username - The username of the FileMaker account.
$password - The password of the FileMaker account.
$sslVarify - (Optional) Can be set to "enable" or "disable". The default is enable.

A Note About SSL Verification
FileMaker Server only accepts Data API connections via https:// which means there's a SSL certificate providing encryption for the connection. cURL will verify that the certificate is valid when making the connection. Your connection will fail if you have the default server certificate or a self-signed certificate installed. To bypass the SSL verification set the value of $sslVarify to "disable".

Example Code:
```
$result = FMXData::fmxConnect('localhost','FMXData','fmxdata','fmxdata','disable');
```

On success, returns a response array with the token and a message array with the status.

```
Array (
     [response] => Array ( [token] => 2a9f528858e9905cf5ea9fdf9281... )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

The Token value is also added to the $GLOBALS variable. You can access it's value by including the global in your code:

Example Code:
```
global $fmx_token;
```

DISCONNECTING FROM FILEMAKER SERVER

I recommend that you disconnect from the session when you have completed all queries. You can choose to leave the FileMaker session open and save the token to the server's session or a cookie. However you'll need to manage the token and verify it hasn't expired with each page load. If necessary, you'll need to refresh the session and keep the token updated. Keep in mind that each open session will be listed in the FileMaker Server admin console on the Clients list until they expire. The maximum number of open sessions is 1000.

fmxDisconnect ( )

Example Code:

```
$result = FMXData::fmxDisconnect( );
```

On success, returns an empty response array and a message array with the status.

```
Array (
     [response] => Array ( )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

FILEMAKER SERVER FUNCTIONS

There are two functions that work directly with FileMaker Server. The fmxProductInfo( ) returns information about FileMaker Server and the fmxListDatabases( ) function returns a list of databases available based on your connection.

fmxProductInfo ( )

Example Code:

```
$result = FMXData::fmxProductInfo( );
```

On success, returns a response array with the product information and a message array with the status.

```
Array (
     [response] => Array ( [productInfo] => Array ( ... ) )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

fmxListDatabases ( )

Example Code:

```
$result = FMXData::fmxListDatabases();
```

On success, returns a response array with the database list and a message array with the status.

```
Array (
     [response] => Array ( [databases] => Array ( ... ) )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

DATABASE FUNCTIONS

There are three database functions to provide access to your database metadata like layout names, script names, and layout information. The fmxListScripts( ) function gets an array of the database scripts. The fmxListLayouts( ) function gets a list of the database layouts. The fmxLayoutMeta( ) function returns layout data including value lists. With the fmxLayoutMeta( ) function you can pass the record ID to return related value lists.

fmxListScripts ( )

Example Code:

```
$result = FMXData::fmxListScripts();
```

On success, returns a response array with the script list and a message array with the status.

```
Array (
     [response] => Array ( [scripts] => Array ( ... ) )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

fmxListLayouts ( )

Example Code:

```
$result = FMXData::fmxListLayouts();
```

On success, returns a response array with the layouts list and a message array with the status.

```
Array (
     [response] => Array ( [layouts] => Array ( ... ) )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

fmxLayoutMeta ($layout {, $recordId })

Parameters:
$layout - The FileMaker Layout to query.
$recordId - (Optional) record id for related value lists.

Example Code:
```
$result = FMXData::fmxLayoutMeta( "CONTACTS", 15297 );
```

On success, returns a response array with multiple arrays for field MetaData, portal MetaData, value lists, etc., and a message array with the status.

```
Array (
     [response] => Array (
          [fieldMetaData] => Array ( ... )
          [portalMetaData] => Array (
               [email_portal] => Array ( ... )
               [phone_portal] => Array ( ... )
          )
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

RECORD FUNCTIONS & OPTIONS

There are six FMXData functions for working with FileMaker records fmxCreateRecord( ), fmxEditRecord( ), fmxDeleteRecord( ), fmxGetRecord( ), fmxGetRange( ), and fmxFindRecords( ). All of the previous functions you could just call the function, there were no additional options you could add. However, the record functions you need to initialize a new instance of the class then you can add additional configuration options and execute the function. Your code might look like something like this:

```
$fmx = new FMXData( );
$fmx->addPostFieldArray($data);
$result = $fmx->fmxCreateRecord('CONTACT');
```

If you need additional options you just add them to the $fmx class object. So for example, if I want to add a FileMaker script, I can add the "addScript( )" option.

```
$fmx = new FMXData( );
$fmx->addPostFieldArray($data);
$fmx->addScript('FMX_Script','param1');
$result = $fmx->fmxCreateRecord('CONTACT');
```

All of the compatible options will be listed with the function information. For the most part it does not matter the order you add your options. However, there is an exception to this when adding sort fields. If you don't specify the $sortOrder parameter the fields will be sorted in the order you add them.

fmxCreateRecord ($layout)

Parameters:
$layout - The FileMaker Layout to query.

Example Code:
```
$fieldArray['First Name'] = 'Duane';
$fieldArray['Last Name'] = 'Weller';
$fieldArray['Job Title'] = 'Lead Developer';
$fmx = new FMXData();
$fmx->addPostFieldArray($fieldArray);
$fmx->addPostField('Company','Excelisys Inc');
$fmx->addScript('FMX_Script','param1');
$result = $fmx->fmxCreateRecord('CONTACT');
```

On success, returns a response array with the new recordId and modId. If you included a script option the script results will also be included.

```
Array (
     [response] => Array (
          [scriptResult] => Script Run - Param: param1
          [scriptError] => 0
          [recordId] => 20303
          [modId] => 0
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

COMPATIABLE OPTIONS
addPostFieldArray ($fieldArray)
addPostField ($fieldName, $fieldValue)
addScript ($scriptName {, $scriptParameter})
addScriptPresort ($scriptName {, $scriptParameter})
addScriptPrerequest ($scriptName {, $scriptParameter})

fmxEditRecord ($layout, $recordId)

Parameters:
$layout - The FileMaker Layout to query.
$recordId - The FileMaker record ID of the record you wish to edit.

Example Code:
```
$fieldArray['Job Title'] = 'FileMaker Developer';
$fieldArray['Website'] = 'https://www.excelisys.com';
$fmx = new FMXData();
$fmx->addPostFieldArray($fieldArray);
$fmx->addPostField('Company','Excelisys Inc');
$fmx->addScript('FMX_Script','param1');
$result = $fmx->fmxEditRecord('CONTACT',$recId);
```

On success, returns a response array with the modId. If you included a script option the script results will also be included.

```
Array (
     [response] => Array (
          [scriptResult] => Script Run - Param: param1
          [scriptError] => 0
          [modId] => 0
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

COMPATIABLE OPTIONS
addPostFieldArray ($fieldArray)
addPostField ($fieldName, $fieldValue)
addScript ($scriptName {, $scriptParameter})
addScriptPresort ($scriptName {, $scriptParameter})
addScriptPrerequest ($scriptName {, $scriptParameter})

fmxDeleteRecord ($layout, $recordId)

Parameters:
$layout - The FileMaker Layout to query.
$recordId - The FileMaker record ID of the record you wish to delete.

Example Code:
```
$fmx = new FMXData();
$fmx->addScript('FMX_Script','param1');
$fmx->addPresortScript('FMX_Presort_Script','param2');
$fmx->addPrerequestScript('FMX_Prerequest_Script','param3');
$result = $fmx->fmxDeleteRecord('CONTACT',$recId);
```

On success, returns an empty response array. If you included a script option the script results will be included.

```
Array (
     [response] => Array (
          [scriptResult] => Script Run - Param: param1
          [scriptError] => 0
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

COMPATIABLE OPTIONS
addScript ($scriptName {, $scriptParameter})
addScriptPresort ($scriptName {, $scriptParameter})
addScriptPrerequest ($scriptName {, $scriptParameter})

fmxDuplicateRecord ($layout, $recordId)

Parameters:
$layout - The FileMaker Layout to query.
$recordId - The FileMaker record ID of the record you wish to duplicate.

Example Code:
```
$fmx = new FMXData();
$fmx->addScript('FMX_Script','param1');
$fmx->addPresortScript('FMX_Presort_Script','param2');
$fmx->addPrerequestScript('FMX_Prerequest_Script','param3');
$result = $fmx->fmxDuplicateRecord('CONTACT',$recId);
```

On success, returns the record ID of the new record. If you included a script option the script results will be included.

```
Array (
     [response] => Array (
          [recordId] => 20382
          [modId] => 0
          [scriptResult] => Script Run - Param: param1
          [scriptError] => 0
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

COMPATIABLE OPTIONS
addScript ($scriptName {, $scriptParameter})
addScriptPresort ($scriptName {, $scriptParameter})
addScriptPrerequest ($scriptName {, $scriptParameter})

fmxGetRecord ($layout, $recordId)

Parameters:
$layout - The FileMaker Layout to query.
$recordId - The FileMaker record ID of the record you wish to return.

Example Code:
```
$fmx = new FMXData();
$fmx->setResponseLayout('CONTACT');
$fmx->addScript('FMX_Script','param1');
$fmx->limitPortals('phone_portal ,email_portal');
$fmx->limitPortalRows('phone_portal',1,2);
$result = $fmx->fmxGetRecord('CONTACT_LIST',15297);
```

On success, returns a response with the script results, dataInfo (which includes the base table name and record counts). The "data" array will have the record information in the first element. The record information will contain arrays of the "fieldData", "portalData", the "portalDataInfo" as well as the recordId and modId.

```
Array (
     [response] => Array (
          [scriptResult] => Script Run - Param: param1
          [scriptError] => 0
          [dataInfo] => Array ( ... )
          [data] => Array (
               [0] => Array (
                    [fieldData] => Array ( ... )
                    [portalData] => Array (
                         [email_portal] => Array ( ... )
                         [phone_portal] => Array ( ... )
                    )
                    [recordId] => 15297
                    [modId] => 4
                    [portalDataInfo] => Array ( ... )
               )
          )
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

COMPATIABLE OPTIONS
setResponseLayout($layout)
limitPortals($portalNames)
limitPortalRows ($portalName, $limit, $offset)
addScript ($scriptName {, $scriptParameter})
addScriptPresort ($scriptName {, $scriptParameter})
addScriptPrerequest ($scriptName {, $scriptParameter})

fmxGetRange ($layout)

Parameters:
$layout - The FileMaker Layout to query.

Example Code:
```
$fmx = new FMXData();
$fmx->setResponseLayout('CONTACT_LIST');
$fmx->addScript('FMX_Script','param1');
$fmx->addSortParam('First Name','ascend',2);
$fmx->addSortParam('Last Name','ascend',1);
$fmx->setLimitOffset(10,50);
$result = $fmx->fmxGetRange('CONTACT');
```

On success, returns a response with the script results, dataInfo (which includes the base table name and record counts). The "data" array will contain the found records. The record information will contain arrays of the "fieldData", "portalData", the "portalDataInfo" as well as the recordId and modId.

```
Array (
     [response] => Array (
          [scriptResult] => Script Run - Param: param1
          [scriptError] => 0
          [dataInfo] => Array ( ... )
          [data] => Array (
               [0] => Array (
                    [fieldData] => Array ( ... )
                    [portalData] => Array ( ... )
                    [recordId] => 16035
                    [modId] => 0
               )
               [1] => Array ( ... )
          )
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

COMPATIABLE OPTIONS
setResponseLayout($layout)
limitPortals($portalNames)
limitPortalRows ($portalName, $limit, $offset)
addScript ($scriptName {, $scriptParameter})
addScriptPresort ($scriptName {, $scriptParameter})
addScriptPrerequest ($scriptName {, $scriptParameter})
addSortParam($fieldName {, $direction, $sortOrder})
setLimitOffset($limit {, $offset})

fmxFindRecords($layout)

Parameters:
$layout - The FileMaker Layout to query.

Example Code:
```
$fmx = new FMXData();
$fmx->setResponseLayout('CONTACT_LIST');
$fmx->addRequestArray(array('Company'=>'Bayer Ltd'));
$fmx->addRequestArray(array('Company'=>'Bayer Group'));
$fmx->addRequestArray(array('Job Title'=>'Teacher'),TRUE);
$fmx->addSortParam('First Name','ascend',2);
$fmx->addSortParam('Last Name','ascend',1);
$fmx->setLimitOffset(10,50);
$result = $fmx->fmxFindRecords('CONTACT');
```

On success, returns a response with the script results, dataInfo (which includes the base table name and record counts). The "data" array will contain the found records. The record information will contain arrays of the "fieldData", "portalData", the "portalDataInfo" as well as the recordId and modId.

```
Array (
     [response] => Array (
          [dataInfo] => Array ( ... )
          [data] => Array (
               [0] => Array ( ... )
               [1] => Array ( ... )
          )
     )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

COMPATIABLE OPTIONS
setResponseLayout($layout)
addRequestArray($requestArray {, $omit })
limitPortals($portalNames)
limitPortalRows ($portalName, $limit, $offset)
addScript ($scriptName {, $scriptParameter})
addScriptPresort ($scriptName {, $scriptParameter})
addScriptPrerequest ($scriptName {, $scriptParameter})
addSortParam($fieldName {, $direction, $sortOrder})
setLimitOffset($limit {, $offset})

SETTING FIELDS IN FILEMAKER

Use the addPostFieldArray( ) and addPostField( ) options to set the fields and field data you want to send to FileMaker. AddPostFieldArray( ) and addPostField( ) functions are compatible with both the fmxCreateRecord( ) and fmxEditRecord( ) functions.

addPostFieldArray($fieldArray)

Parameters:
$fieldArray - An array of field names and the associated values.

The addPostFieldArray requires an array of $key=>$value pairs where the $key will be the field name and the $value will be the data you want in the field. Usually this data comes from a form on a web page. You can assemble the array using the array( ) function or you can set each element of the array one at a time.

Example Code:
```
$fieldArray = array(
     'Job Title' => 'FileMaker Developer',
     'Website' => 'https://www.excelisys.com'
);
$fmx->addPostFieldArray($fieldArray);
```

OR

```
$fieldArray['Job Title'] = 'FileMaker Developer';
$fieldArray['Website'] = 'https://www.excelisys.com';
$fmx->addPostFieldArray($fieldArray);
```

addPostField($fieldName, $fieldValue)

Parameters:
$fieldName - The name of the field you want to set in FileMaker.
$fieldValue - The value you want to set the associated field to.

With addPostField( ) you can just add the field and value directly. You don't need to create an array.

Example Code:
```
$fmx->addPostField('Company','Excelisys Inc');
```

NOTE: You can use both addPostFieldArray( ) and addPostField( ) at the same time. All of the fields will be sent to FileMaker. You can add either of these options as many times as you need.

COMPATIABLE FUNCTIONS
$fmxCreateRecord ($layout)
$fmxEditRecord ($layout, $recordId)

PERFORMING FILEMAKER FINDS

The addRequestArray( ) option is used exclusively to add FileMaker find requests to the fmxFindRecords( ) functions.

addRequestArray($requestArray {, $omit })

Parameters:
$requestArray - An array of field names and the associated search criteria.
$omit - (Optional) Can be either TRUE or FALSE. Set this to TRUE to mark the request to omit the criteria.

The $requestArray parameter requires an array of $key=>$value pairs where the $key will be the field name and the $value will be the search criteria. You can use FileMaker search operators such as "=", "<", "<=", ">", '>=", "//", "*" for your find criteria.

Example Code:
```
$fmx->addRequestArray(array('Company'=>'Bayer Ltd'));
$fmx->addRequestArray(array('Company'=>'Bayer Group'));
$fmx->addRequestArray(array('Job Title'=>'Teacher'),TRUE);
```

COMPATIABLE FUNCTIONS
fmxFindRecords($layout)

SORTING THE RESULTS

Use the addSortParam( ) option to indicate how you want the results sorted. The direction parameter indicates if you want the field sorted ascending or descending. If you don't specify a direction "ascending" is the default. Use the sortOrder parameter to indicate which order to sort the field, 1, 2, 3, etc. If you do not indicate a sort order then the default is to use the order in which they are added. You can use as many addSortParam( ) options as you need.

addSortParam($fieldName {, $direction, $sortOrder})

Parameters:
$fieldName - The name of the field to sort.
$direction - (Optional) Can be either "ascend" or "decend".
$sortOrder - (Optional) Integer beginning with 1 for the first addSortParam( ) used, 2 for the second, etc.

Example Code:
```
$fmx->addSortParam('First Name','ascend',2);
$fmx->addSortParam('Last Name','ascend',1);
```

COMPATIABLE FUNCTIONS
fmxGetRange ($layout)
fmxFindRecords($layout)

LIMITING THE FIND RESULTS

Use the setLimitOffset( ) option to limit the result set. The default limit for records is 100. The default offset is 1.

setLimitOffset($limit {, $offset })

Parameters:
$limit - The maximum number of records to return.
$offset - (Optional) The number of the first record to return (records before it are skipped).

Example Code:
```
$fmx->setLimitOffset(10,50);
```

COMPATIABLE FUNCTIONS
fmxGetRange ($layout)
fmxFindRecords($layout)

WORKING WITH FILEMAKER SCRIPTS

You can execute FileMaker Scripts in just about all the fmxData record functions. There are three options to trigger FileMaker Scripts. The three options are addScriptPrerequest( ), addScriptPresort( ), and addScript( ). They work in this order...

addScriptPrerequest( ) - Executes the script before the find is run.

[The find is run]

addScriptPresort( ) - Executes after the find but before and sorts are run.

[The sorts are run]

addScript( ) - Executes at the end of the request.

addScriptPrerequest($scriptName {, $scriptParameter })
addScriptPresort($scriptName {, $scriptParameter })
addScript($scriptName {, $scriptParameter })

Parameters:
$scriptName - The name of the script you want to trigger.
$scriptParameter - (Optional) The script parameter you want to pass to the script.

Example Code:
```
$fmx->addPrerequestScript('FMX_Prerequest_Script','param3');
$fmx->addPresortScript('FMX_Presort_Script','param2');
$fmx->addScript('FMX_Script','param1');
```

NOTE: You can use all three at the same time, but you can only use them once each.

COMPATIABLE FUNCTIONS
fmxCreateRecord ($layout)
fmxEditRecord ($layout, $recordId)
fmxDeleteRecord ($layout, $recordId)
fmxGetRecord ($layout, $recordId)
fmxGetRange ($layout)
fmxFindRecords($layout)

SETTING THE RESPONSE LAYOUT

For many functions you can use the setResponseLayout( ) option to determine which layout your query is returned from. For example, you can use the fields on a detail layout to perform a complex find, but set the response layout to a list view that may have a limited number of fields on it.

setResponseLayout($layout)

Parameters:
$layout - The FileMaker Layout to query.

Example Code:
```
$fmx->setResponseLayout('CONTACT_LIST');
```

COMPATIABLE FUNCTIONS
fmxGetRecord ($layout, $recordId)
fmxGetRange ($layout)
fmxFindRecords($layout)

WORKING WITH PORTALS

There are two options to help you work with portals, limitPortals( ) and limitPortalRows( ). Using limitPortals( ) you can specify which portals to return. The limitPortalRows( ) option allows you to set a limit and offset for each portal.

limitPortals($portalNames)

Parameters:
$portalNames - A comma delimited list of the portal object names from the layout in FileMaker.

Example Code:
```
$fmx->limitPortals('phone_portal, email_portal');
```

limitPortalRows ($portalName, $limit {, $offset })

Use limitPortalRows( ) to limit the number of portal records returned. The default limit for portals is 50. The default offset is 1.

Parameters:
$portalName - The portal object names from the layout in FileMaker.
$limit - The number of records to return from the portal.
$offset - (Optional) The number of the first record to return (records before it are skipped).

Example Code:
```
$fmx->limitPortalRows('phone_portal',1,2);
```

COMPATIABLE FUNCTIONS
fmxGetRecord ($layout, $recordId)
fmxGetRange ($layout)
fmxFindRecords ($layout)

WORKING WITH CONTAINER FIELDS

There are now two functions to help work with container fields. The fmxUploadFile( ) function will upload a file into a container field. The fmxContainerData( ) function retrieves container field data. Both functions are static functions that can operate outside of the fmxConnect( )... fmxDisconnect( ) session. However, the fmxContainerData( ) function was designed to work with fmxConnect( ) to support the required cookie for Authenticated Streaming (more on this below).

Uploading Files Into Container Fields
It's important to note that the $pathToFile parameter must be a full file path. Don't get this confused with a URL, this is the path to the file relative to the root of the disk drive. You can use the magic constant __DIR__ to get the full path to the directory of the current PHP file, then you can just append the path to your upload file from there.

Also note that the $repetition parameter is now optional but in order to comply with PHP 8 function structure requirements $pathToFile was moved to the first parameter.

fmxUploadFile($pathToFile, $layout, $recordId, $fieldName {, $repetition })

Parameters:
$pathToFile - The full path to the file that needs to be uploaded.
$layout - The FileMaker Layout to query.
$recordId - The FileMaker record ID of the record you wish to edit.
$fieldName - The name of the container field you wish to upload to.
$repetition - (Optional) The number of the field repetition. When omitted this defaults to 1.

Example Code:
```
$path = __DIR__.'/photo.jpg';
$upload = FMXData::fmxUploadFile($path,'CONTACT',15297,'Photo',1);
```

On success, returns a response array with the modId of the record and a message array with the status.

```
Array (
     [response] => Array ( [modId] => 3 )
     [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

Downloading Container Data
It used to be that you could just use the URLs for container fields as is. However, in FileMaker Server 19 Claris added Authenticated Streaming for container downloads. By default Authenticated Streaming is enabled when you install FileMaker Server. If you disable Authenticated Streaming you can use the container data URLs like you did before but if Authenticated Streaming is enabled you must use a function like fmxContainerData( ) to successfully download the container data.

To manage Authenticated Streaming you can run the following from the command line on the FileMaker Server:

To get the current streaming setting run:
```
fmsadmin get serverprefs AuthenticatedStream
```

To disable Authenticated Streaming run:
```
fmsadmin set serverprefs AuthenticatedStream=2
```

To enable Authenticated Streaming run:
```
fmsadmin set serverprefs AuthenticatedStream=1
```

More information can be found in this Claris article:
https://community.claris.com/en/s/article/Allowing-Claris-Connect-to-access-container-data-on-a-FileMaker-host

fmxContainerData($url)

Parameters:
$url - The container data URL returned in the Data API data for the record.

Example Code:

```
// connect to FileMaker Server first - this sets the required cookie
FMXData::fmxConnect('localhost','FMXData','fmxdata','fmxdata','disable');

// get the url of the container field from a previous query of the record
$url = 'https://192.168.4.102/Streaming/MainDB/
4C38E09F2DE32755AC457BEB414E17F9E9F5A06DD18EA3816D74CB96200438B1.jpg?
RCType=EmbeddedRCFileProcessor';

// download the container field data
$download = FMXData::fmxContainerData($url);

// now disconnect
FMXData::fmxDisconnect();

< do something with the download >
```

On success, this function returns the data of the Container Field file.

SETTING GLOBAL FIELDS

You can use fmxSetGlobalFields( ) and addGlobalField( ) to set global fields in the database. Use this to set globals before running a script or performing a find. The global field values will persist until you run fmxDisconnect( ) and terminate the session. Use the addGlobalField( ) option to specify the fields and values you want to set. You can use as many addGlobalField( ) options as you need.

fmxSetGlobalFields( )

Example Code:

```
$fmx = new FMXData();
$fmx->addGlobalField('CONTACT','global1','FileMaker');
$fmx->addGlobalField('CONTACT','global2','Rocks');
$result = $fmx->fmxSetGlobalFields();
```

On success, returns an empty response array and a message array with the status.

```
Array (
        [response] => Array ( )
        [messages] => Array ( [0] => Array ( [code] => 0 [message] => OK ) )
)
```

addGlobalField($baseTable, $fieldName, $value)

Parameters:
$baseTable - The base table name in FileMaker.
$fieldName - The name of the field you wish to set.
$value - The value you want to set the field to.